

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-acc7e4af64428952e.jsonl`  
- SHA-256: `001628f405279e3d3d1685402f0e94f49b21a12a9c5b95715b0fee718e431880`  
- Source modified: `2026-07-02T03:00:38+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T02:58:05.139Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.

A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer:

```
{  
  "title": "Path-jail rejects gs://,s3://,gdrive-api:// before scheme resolution ? bucket ingest impossible on the exposed alpha box (issue #327 still open)",  
  "category": "tech-debt",  
  "severity": "P1",  
  "file": "backend/main.py",  
  "line": 1376,  
  "evidence": "main.py:1376 calls `validate_input_path(req.video_path, allowed_root=allowed_root)` BEFORE `storage.resolve_input_ref` at main.py:1379; backend/utils/paths.py:45-55 `resolve_under_root` realpaths the raw URI so `gs://bucket/x.mp4` escapes the jail -> 400. Startup check startup.py:106-119 makes the jail mandatory when exposed (EXPOSED_NO_VIDEO_ROOT), so an exposed engine can never ingest from a bucket. gh: issue #327 OPEN (verified 2026-07-02), proposing scheme-aware validation.",  
  "why_it_matters": "Blocks the alpha CUJ (large-video ingest via gs:// instead of 500 MB browser uploads) and is a stale-open thread since before 06-26; the fix must be done carefully to avoid reopening the arbitrary-local-file hole the jail exists for.",  
  "proposed_fix": "Implement #327 option (b): jail only after resolve_input_ref, applying resolve_under_root exclusively to backend=='local' refs; keep traversal/symlink/absolute rejection for local paths and add both allow (bucket URI accepted) and reject (local traversal still 400) tests, updating tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri.",  
  "effort": "M",  
  "tracked_in": "https://github.com/Khelsutra/badminton-highlight-indexer/issues/327"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you

personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T02:58:15.245Z)

```
1330         if paths and isinstance(response, dict):
1331             response["reports"] = paths
1332
1333
1334     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1335         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
1336
1337         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
1338         that needs a
1339         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
1340         guard degrades
1341         to a no-op rather than blocking a job it cannot measure."""
1342     try:
1343         sc = (
1344             storage_config.model_dump()
1345             if hasattr(storage_config, "model_dump")
1346             else (storage_config or {})
1347         )
1348         backend = storage.get_storage(input_ref.backend, config=sc)
1349         return int(backend.stat(input_ref).size)
1350     except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
1351         not block
1352         logger.debug(
1353             "free-space: could not stat remote source %s: %s", input_ref.uri, e
1354         )
1355     return None
1356
1357
1358 @app.post("/api/process", status_code=202)
1359 def process_video(req: ProcessRequest, request: Request):
1360     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
1361
1362     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
1363     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
1364     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
1365     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
1366     """
1367     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
1368     single-user,
1369     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
1370     missing or
1371     # spoofed identity header fails here with 401, before any work) and tags the job's
1372     owner_id.
1373     try:
1374         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1375     except edge_auth.EdgeAuthError as e:
1376         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1377
1378     allowed_root = getattr(
1379         getattr(request.app.state.config, "security", None), "allowed_video_root", None
1380     )
1381     try:
1382         validate_input_path(req.video_path, allowed_root=allowed_root)
1383         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
1384         unsupported
1385         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
1386         input_ref = storage.resolve_input_ref(
1387             req.video_path, getattr(request.app.state.config, "storage", None)
1388         )
1389     except ValueError as e:
```

```

1383         raise HTTPException(status_code=400, detail=str(e)) from e
1384     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
1385     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
1386     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
1387     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
1388     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
1389     # rejects a non-local URI as out-of-root.
1390     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1391         raise HTTPException(
1392             status_code=404, detail=f"Video file not found at '{req.video_path}'"
1393         )
1394     # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
1395     # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
1396     # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
1397     # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
1398     if input_ref.backend != "local":
1399         need = _remote_input_size_bytes(
1400             input_ref, getattr(request.app.state.config, "storage", None)
1401         )
1402         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
1403     if req.segmenter and not segmenter_registry.get(req.segmenter):
1404         available = list(segmenter_registry.list_available().keys())
1405         raise HTTPException(
1406             status_code=400,
1407             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
1408         )
1409
1410     job_id = uuid.uuid4().hex
1411     if not request.app.state.db.create_job(
1412         job_id,
1413         req.video_path,
1414         req.segmenter,
1415         req.player_pool,
1416         req.max_frames,
1417         owner_id=owner_id,
1418         locale=req.locale,
1419         compute=req.compute,

```

3. user (2026-07-02T02:58:15.408Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3     import os
4     from typing import Any, Optional
5
6
7     def output_base(config: Any) -> str:
8         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
9         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
10         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
11         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
12

```

```

13         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
14         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
15         (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``output``. """
16         try:
17             out = (
18                 config.output.output_dir
19                 if not isinstance(config, dict)
20                 else config.get("output", {}).get("output_dir")
21             )
22         except Exception:
23             out = None
24         return out or "output"
25
26
27     def safe_output_filename(name: str) -> str:
28         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
29
30         `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
31         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
32         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
33         """
34         if not name or not name.strip():
35             raise ValueError("output_filename must not be empty")
36         if "\x00" in name:
37             raise ValueError("output_filename must not contain a null byte")
38         if "/" in name or "\\" in name:
39             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
40         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
41             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
42         return name
43
44
45     def resolve_under_root(path: str, root: str) -> str:
46         """Return the real path, asserting it is contained under `root` (raises ValueError
if not). """
47         rp = os.path.realpath(path)
48         rr = os.path.realpath(root)
49         try:
50             contained = os.path.commonpath([rp, rr]) == rr
51         except ValueError: # different drives on Windows
52             contained = False
53         if not contained:
54             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
55         return rp
56
57
58     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
59         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
60
61         With no allowed_root (default, single-user local mode) only the null-byte check
applies,
62         so the local 'point it at any of my videos' workflow is preserved. Setting an
63         allowed_root (a future hosted/tenant deployment) enforces containment.
64         """
65         if "\x00" in path:
66             raise ValueError("video_path must not contain a null byte")
67         if allowed_root:
68             resolve_under_root(path, allowed_root)
69

```

4. user (2026-07-02T02:58:22.093Z)

No files found

5. user (2026-07-02T02:58:23.529Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
959c630 fix(#397): write per-video report to the resolved local path, not the raw URI (#400)
b6214e5 chore: ruff format the whole repo (259 files) (#387)
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)
9cd7598 fix(tests): resolve remaining test signature mismatches from app-factory refactor (#371)
de09ec8 refactor(main): app-factory + typed-config migration (D2/D3/P2/P3/P5/P7) (#370)
5b71af8 D3-c: remove AppConfig .get() shim, flip extra='forbid' (#367)
b25fdb8 docs: explain lazy cv2 import in --debug-clip path (Q8) (#361)
f523d3a refactor: enable isort (I) in ruff, auto-fix 143 files (Q6) (#360)
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

6. user (2026-07-02T02:58:30.383Z)

```
badminton-highlight-indexer\backend\config\startup.py:114:
"EXPOSED_NO_VIDEO_ROOT",
badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\README.md:170:[Omitted long matching
line]
badminton-highlight-indexer\tests\test_startup_checks.py:336:     assert _codes(checks) ==
{"EXPOSED_NO_VIDEO_ROOT"}
badminton-highlight-indexer\tests\test_startup_checks.py:345:     assert "EXPOSED_NO_VIDEO_ROOT"
in _codes(
badminton-highlight-indexer\tests\test_startup_checks.py:410:     assert codes ==
{"EXPOSED_NO_VIDEO_ROOT", "EXPOSED_EDGE_AUTH_INCOMPLETE"}
badminton-highlight-indexer\tests\test_startup_checks.py:426:     assert codes ==
{"EXPOSED_NO_VIDEO_ROOT", "STORAGE_INCOHERENT"}
```

7. user (2026-07-02T02:58:30.804Z)

badminton-highlight-indexer\backend\config\startup.py

8. user (2026-07-02T02:58:35.905Z)

```
90         return False
91
92
93     def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
94         """Exposure-safety posture for an *exposed* HTTP server. Gated on
95         ``security.edge_auth_enabled``
96         (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ?
97         the "I am
98         exposing this engine behind the CF-Access gate" signal ? the gate alone isn't
99         enough: verify the
100         rest of the posture at boot so a half-configured exposure fails fast instead of
101         leaking / 500-ing
102         per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env``
103         is the
104         caller's environment view (so the host-allowlist check is testable)."""
105         sec = getattr(config, "security", None)
106         if not sec or not getattr(sec, "edge_auth_enabled", False):
107             return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical
```

```

to today).
103
104     out: list[StartupCheck] = []
105
106     # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
WITHOUT it,
107     # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
= "any local
108     # file", the single-user default). Fatal for an exposed engine.
109     root = getattr(sec, "allowed_video_root", None)
110     if not root or not str(root).strip():
111         out.append(
112             StartupCheck(
113                 LEVEL_ERROR,
114                 "EXPOSED_NO_VIDEO_ROOT",
115                 "security.edge_auth_enabled is ON (engine exposed) but
security.allowed_video_root is "
116                 "unset ? an authenticated tenant could make /api/process read an
arbitrary local file. "
117                 "Set allowed_video_root to a jail directory before exposing the
engine.",
118             )
119         )
120
121     # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only
fails on the
122     # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never
starts
123     # "healthy" then 401s every request.
124     team = (getattr(sec, "cf_team_domain", "") or "").strip()
125     aud = (getattr(sec, "cf_access_aud", "") or "").strip()
126     if not team or not aud:
127         missing = " + ".join(
128             n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if not v
129         )
130         out.append(
131             StartupCheck(
132                 LEVEL_ERROR,
133                 "EXPOSED_EDGE_AUTH_INCOMPLETE",
134                 f"security.edge_auth_enabled is ON but {missing} is not configured ? the
Cf-Access JWT "
135                 "cannot be verified (every request would fail). Set
security.cf_team_domain "
136                 "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud
(the CF Access "
137                 "application AUD tag).",
138             )
139         )

```

9. user (2026-07-02T02:58:50.467Z)

```

badminton-highlight-indexer\tests\test_api_endpoints.py:809: Counterweight =
test_process_hosted_mode_rejects_nonlocal_uri (gs:// -> 400 when allowed_video_root
badminton-highlight-indexer\tests\test_api_endpoints.py-810- is set). NOTE: the
m.storage.fetch fake proves the CODE routing, not the live GCS download (that
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-811- needs google-cloud-storage + ADC on the box ?
verified at deploy time, not in this offline suite)."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-812- client, db, tmp_path, mp = env
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-813- payload = b"bucket-fetched-bytes-payload"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-814-
badminton-highlight-indexer\tests\test_api_endpoints.py-815- def fake_fetch(uri_, dst=None,

```

```

*, config=None, overwrite=False):
badminton-highlight-indexer\tests\test_api_endpoints.py-816-         with open(dst, "wb") as fh:
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-817-             fh.write(payload)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-818-             return dst
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-819-
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-820-         mp setattr(m.storage, "fetch", fake_fetch)
badminton-highlight-indexer\tests\test_api_endpoints.py-821-         mp setattr(m.registry,
"run_all_with_context", lambda *a, **k: (_pass(), {}))
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-822-         mp setattr(
badminton-highlight-indexer\tests\test_api_endpoints.py-823-             m.segmenter_registry, "get",
lambda name: _FakeSeg
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-824-         ) # _FakeSeg persists a play_segment
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-825-
badminton-highlight-indexer\tests\test_api_endpoints.py-826-         r = client.post("\api\process",
json={"video_path": uri, "segmenter": "x"})
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-827-         assert r.status_code == 202 # non-local skips the
os.path.exists 404 gate
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-828-         job =
client.get(f"/api/jobs/{r.json()['job_id']}").json()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-829-         assert job["status"] == "done" and
job["result"]["status"] == "success"
--
badminton-highlight-indexer\tests\test_api_endpoints.py:903: def
test_process_hosted_mode_rejects_nonlocal_uri(env):
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-904-     """Hosted mode (allowed_video_root set) must reject
a bucket/Drive URI ? a tenant can't point
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-905-     the engine at an arbitrary bucket. Pins the security
contract instead of an incidental side effect."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-906-     client, db, tmp_path, mp = env
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-907-     root = tmp_path / "tenant"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-908-     root.mkdir()
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-909-     mp setattr(m.global_config.security,
"allowed_video_root", str(root))
badminton-highlight-indexer\tests\test_api_endpoints.py-910-     for bad in ("gs://b/x.mp4",
"s3://b/x.mp4", "gdrive://Sharing/x.mp4"):
badminton-highlight-indexer\tests\test_api_endpoints.py-911-         r =
client.post("\api\process", json={"video_path": bad})
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-912-         assert r.status_code == 400, f"{bad} ->
{r.status_code}"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-913-         # A local file UNDER the root is still accepted
(passes validation; processing mocked elsewhere).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-914-         under = root / "ok.mp4"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-915-         under.write_bytes(b"hashable")
badminton-highlight-indexer\tests\test_api_endpoints.py-916-         mp setattr(m.registry,
"run_all_with_context", lambda *a, **k: (_pass(), {}))

```

```

badminton-highlight-indexer\tests\test_api_endpoints.py-917- mp setattr(m.segmenter_registry,
"get", lambda name: _FakeSeg)
badminton-highlight-indexer\tests\test_api_endpoints.py-918- r = client.post("\api\process",
json={"video_path": str(under), "segmenter": "x"})
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-919- assert r.status_code == 202
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-920-
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-
indexer\tests\test_api_endpoints.py-921-
badminton-highlight-indexer\tests\test_api_endpoints.py-922-def
test_cli_nonlocal_input_fetches_and_keeps_uri(tmp_path, monkeypatch):
badminton-highlight-indexer\tests\test_api_endpoints.py-923- """The CLI --video-path block (a
second M2 call-site) routes a bucket URI through fetch: the

```

10. user (2026-07-02T02:58:53.070Z)

```

{"createdAt":"2026-06-22T13:54:42Z","labels":[],"number":327,"state":"OPEN","title":"Allow gs://
/s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt recognized storage
backends from the path-jail","updatedAt":"2026-06-22T13:54:42Z"}

```

11. assistant (2026-07-02T02:59:10.432Z)

All core claims verified. Let me quickly sanity-check the "500 MB browser upload" claim that underpins the P1 severity.

12. user (2026-07-02T02:59:13.160Z)

```

badminton-highlight-indexer\backend\providers\gemini.py:23:MAX_UPLOAD_MB = 500
badminton-highlight-indexer\backend\providers\gemini.py:129:         if size_mb > MAX_UPLOAD_MB:
badminton-highlight-indexer\backend\providers\gemini.py:131:             f"[{log_prefix}]
{video_path} is {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap ? "
badminton-highlight-indexer\backend\providers\gemini.py:135:             metrics["error"] =
f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
badminton-highlight-indexer\backend\config\models.py:226:     #
(backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
badminton-highlight-indexer\backend\config\models.py:339:     max_upload_mb: int = 4096
badminton-highlight-indexer\backend\reporting\spec.yaml:42: # The provider REFUSES clips over
its upload cap (providers/gemini.py::MAX_UPLOAD_MB)
badminton-highlight-indexer\backend\api\uploads.py:51:     max_bytes = int(getattr(sec,
"max_upload_mb", 4096) or 4096) * 1024 * 1024
badminton-highlight-indexer\backend\pipeline\segmenters\gemini.py:650:         # 4K chunks
(~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
badminton-highlight-indexer\docs\CODE_MAP.md:115:[Omitted long matching line]
badminton-highlight-indexer\docs\CODE_MAP.md:150:[Omitted long matching line]
badminton-highlight-indexer\docs\CODE_MAP.md:211:[Omitted long matching line]
badminton-highlight-indexer\docs\CODE_MAP.md:223:[Omitted long matching line]
badminton-highlight-indexer\docs\CODE_MAP.md:384:| `test_providers.py` /
`test_provider_gemini.py` | `providers` registry + GeminiProvider lifecycle (`MAX_UPLOAD_MB`
guard refuses LOUDLY w/ `oversize_skipped` metric, non-list JSON -> [], cleanup delete) |
badminton-highlight-indexer\docs\VIDEO_LOCALITY_MODEL.md:20:rejects anything over
`security.max_upload_mb` (default **4096 MB**), and even

```

[Showing results with pagination = limit: 15]

13. assistant (2026-07-02T02:59:48.828Z)

Let me check the docs' stance on the upload path and bucket ingest to assess severity honestly.

14. user (2026-07-02T02:59:50.746Z)

```

1     # Video locality ? where the bytes live (mental model + open questions)
2
3     **Status:** THINKING DOC (owner-requested, 2026-06-12) ? a shared mental model for

```



```

4 the upload/storage problem, with the open questions named. **No implementation is
5 greenlit by this doc** (per CLAUDE.md, both tracks stay owner-gated).
6 **Companions:** `UI_PROPOSAL.md` (B2/B3 upload/download), `HOSTING_PLAN.md`,
7 `PLATFORM_ARCHITECTURE.md` §3 (future StorageBackend/ComputeBackend registries),
8 collector ADR-002 (the frame-identical proxy contract this doc leans on).
9
10 ## 0. The questions that triggered this (and the short answers)
11
12 ***"Do we store the video they upload?"** Yes ? today the chunked-upload flow
13 assembles the full file under `security.upload_dir` (default `output/uploads`) on
14 our box, registers it by MD5, and nothing ever deletes it (retention policy is an
15 acknowledged gap ? BACKLOG). Processing also strews derived artifacts (chunks,
16 frames, trajectories, WSL temp) and, on the Gemini path, uploads ?1280px
17 re-encoded chunks to Google's Files API.
18
19 ***"How would they upload a 10 GB file?"** Today they can't: `/api/upload/init`
20 rejects anything over `security.max_upload_mb` (default **4096 MB**), and even
21 under the cap the file moves as sequential ?90 MB parts through a free-tier edge
22 whose body limit forced that part size. For scale: 10 GB ? 30?45 min of 4K phone
23 video (or ~2?3 h at 1080p); on a typical Indian uplink that's **~1 h at 20 Mbps,
24 ~4.5 h at 5 Mbps** ? before retries. Browser-tab uploads of that size are
25 miserable everywhere; expecting academies to manage S3/GCS buckets is a
26 non-starter. **The 10 GB problem is real and the current design doesn't solve it.**
27
28 ***"Can we make it so there is NO upload?"** Mostly ? because of §1.
29
30 ## 1. The central insight: timestamps are the product; the video is dead weight
31
32 Walk the pipeline and ask what each stage actually consumes:
33
34 - The **Gemini segmenter** never sees the original ? it re-encodes chunks down to
35 **1280 px** (`indexing.ai_chunk_scale_width`) before uploading them to Google.
36 - The **detectors/featurizer** consume downscaled frames and trajectories.
37 - The only consumer of the original full-res bytes is the **highlight compiler**,
38 and only to cut clips ? a pure-ffmpeg operation that can run **wherever the
39 original lives**.
40 - What the customer receives is `{rally intervals, report, reel}`. Intervals +
41 report are **kilobytes**. Given the intervals, the reel can be cut on the
42 machine that already has the original.
43
44 So: **any design that moves the 10 GB is moving the wrong thing.** The question
45 is never "how do we upload the video" but "what is the smallest artifact that
46 must cross the network for this tier of service."
47
48 ## 2. The locality ladder
49
50 From most to least bytes leaving the academy's Windows desktop. Sizes are rough
51 ratios for a 4K source ? measure on a real file before committing to any tier.
52
53 | | What crosses the network | ~Bytes (10 GB source) | What it needs | Status |
54 |---|---|---|---|---|
55 | **L3 ? full upload** (today's flow) | everything | 10 GB (blocked: 4 GB cap) | paid
56 edge tier + resumable uploads + server disk + retention policy | built, capped, wrong for big
57 files |
58 | **L2 ? proxy upload** | a timeline-identical ?1080p re-encode | ~1?2.5 GB | a tiny
59 client prep step (ffmpeg) ? **the exact frame-identical contract we already built and verified
60 as collector `prep-annotation-proxy`** (same fps, same frame count, parity-checked, MD5?MD5
61 manifest) | contract proven; client wrapper + server acceptance = new work |
62 | **L1 ? chunks/candidates upload** | only the ?1280px AI chunks (or just candidate
63 windows) the Gemini path needs anyway | ~0.3?1 GB | client-side chunk+downscale (ffmpeg, CPU);
64 server becomes a **GPU-less orchestrator** | the server half exists (it's the gemini segmenter's
65 own flow, relocated) |
66 | **L1-direct** | chunks go **straight from the client to Gemini** ? zero video bytes
67 through our server | ~0 through us | per-user key custody / metering / cost ceilings (OQ4) |
68 idea only |

```

59 | ****L0 ? fully local**** | nothing (calls to Google only if the Gemini path is used) | 0 |
the owned on-device model (roadmap north star) or a local GPU + our stack | not alpha-feasible;
the long-term answer |
60 | ****L4 ? sneakernet**** | a hard drive | 0 over network | a person (the field associate /
owner) | how the pilot actually works today |

15. user (2026-07-02T02:59:52.619Z)

Today

When ``security.allowed_video_root`` is set (required to expose), ``/api/process`` runs
``validate_input_path`` (local-under-jail) ****before**** resolving the storage scheme, so
``gs://bucket/x.mp4`` ? ``os.path.realpath`` escapes the jail ? ****400****. Bucket ingest is therefore
impossible on the exposed alpha box. Pinned by
``tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri`` (gs/s3/gdrive all
400).

Want

The owner wants to ingest large videos via ``gs://`` on the hosted box (far more convenient than a
500 MB browser upload).

Proposed

Make ``validate_input_path`` / the ``/api/process`` preamble ****scheme-aware****: either (a) allow
recognized storage schemes (``gs://``, ``s3://``, ``gdrive-api://``) ****before**** the local-jail check, or
(b) apply the jail ****only to local refs**** (validate after ``storage.resolve_input_ref``, jailing
only ``backend == "local"``).

? Must NOT reopen the arbitrary-local-file hole ? the jail must still fully apply to ****local****
paths (traversal/symlink/absolute). Add tests for both the allow (bucket URI accepted) and the
reject (local traversal still 400) paths. Pairs with the ``gs://`` ingest the alpha CUJ +
DEPLOY_ALPHA §7 currently disclaim.

16. user (2026-07-02T03:00:38.256Z)

Structured output provided successfully